

Презентация по лабораторной работе №5

Аппарат сетей Петри. Задача «Обедающие философы»

Малюга Валерия Васильевна

2026-04-18

- 1 Цели и задачи
- 2 Задание
- 3 Теоретическое введение
- 4 Анализ результатов
- 5 Выводы
- 6 Список литературы

- **Малюга Валерия Васильевна**
- Студент НФИбд-01-23
- Российский университет дружбы народов им. П. Лумумбы
- <https://vvmalyuga.github.io/ru/>
- 1132236050@rudn.ru

Раздел 1

Цели и задачи

- 1 Построить сеть Петри для пяти философов, моделируя захват и освобождение вилок.
- 2 Обнаружить состояние взаимной блокировки (deadlock), когда каждый философ взял одну вилку и ждёт вторую.
- 3 Провести имитационное моделирование (стохастическое и детерминированное) и выявить наличие deadlock.
- 4 Модифицировать сеть, чтобы предотвратить deadlock.
- 5 Проанализировать результаты и оформить отчёт с графиками и анимацией.

Раздел 2

Задание

- Создать рабочий каталог, установить необходимые пакеты
- Выполнить предложенный код
- Преобразовать код в литературный стиль
- Сгенерировать производные форматы
- Интегрировать документацию в отчёт
- Добавить вычисления для набора параметров
- Выложить результаты на GitHub и GitVerse

Раздел 3

Теоретическое введение

Сеть Петри была предложена Карлом Адамом Петри в 1962 году [2]. Это математический аппарат для моделирования дискретных систем. Графически она представляется как двудольный ориентированный граф двух типов вершин: позиции (круги) и переходы (прямоугольники).

- Позиции (Places) суть пассивные элементы, описывающие состояние системы (например, наличие ресурса или выполнение условия). Внутри позиции могут находиться фишки (tokens) — неотрицательное целое число, указывающее на количество ресурсов.
- Переходы (Transitions) суть активные элементы, описывающие события или действия системы. Они могут срабатывать, изменяя состояние модели.
- Дуги (Arcs) суть направленные соединения между позициями и переходами (но не между двумя позициями или двумя переходами), которые показывают, как состояние влияет на события и наоборот.
- Маркировка (Marking) есть распределение фишек по позициям в определённый момент времени, то есть текущее состояние модели. Смена маркировок происходит при срабатывании переходов в соответствии с определёнными правилами.

Классическая постановка

За круглым столом сидят N философов. Перед каждым философом стоит тарелка с едой. Между каждыми двумя соседними философами лежит одна вилка (или палочка для еды). Таким образом, количество вилок равно количеству философов. Задача была сформулирована Эдсгером Дейкстрой в 1965 году как иллюстрация проблемы синхронизации в параллельных вычислительных системах [1].

Основные положения

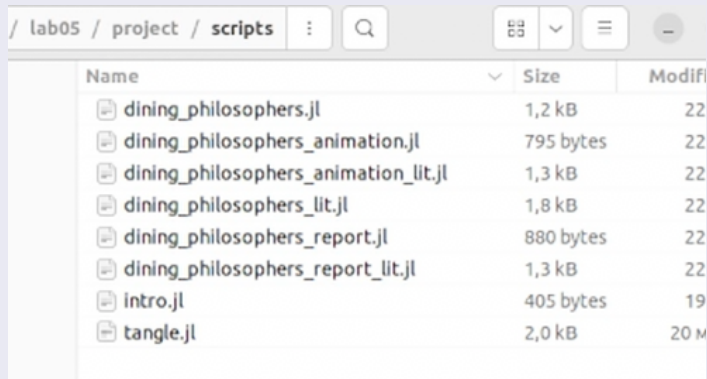
1 Правила поведения философов:

- Философ может находиться в одном из трёх состояний: думает, голоден (хочет есть), ест.
- Чтобы поесть, философу необходимы две вилки — та, что слева от него, и та, что справа.
- Если философ не может получить обе вилки одновременно, он ждёт, пока они освободятся.
- Поев, философ кладёт обе вилки обратно на стол и возвращается к размышлениям.

2 Ограничения:

- Вилками нельзя пользоваться одновременно двум философам.
- Философ не может отнять вилку у соседа — только дожждаться, пока тот её положит.

Структура проекта и модуль



/ lab05 / project / scripts			
Name	Size	Modifi	
dining_philosophers.jl	1,2 kB	22	
dining_philosophers_animation.jl	795 bytes	22	
dining_philosophers_animation_lit.jl	1,3 kB	22	
dining_philosophers_lit.jl	1,8 kB	22	
dining_philosophers_report.jl	880 bytes	22	
dining_philosophers_report_lit.jl	1,3 kB	22	
intro.jl	405 bytes	19	
tangle.jl	2,0 kB	20 M	

Рисунок 1: Структура каталогов проекта

Каталоги src/, scripts/, data/, plots/ организованы по DrWatson.

Классическая сеть (без арбитра)

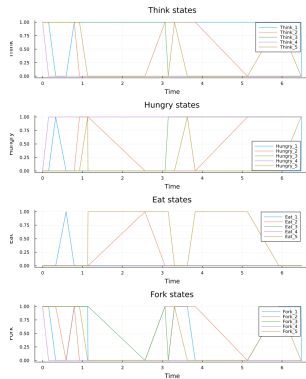


Рисунок 2: Динамика классической сети

Четыре панели: Think, Hungry, Eat, Fork. Со временем все философы голодны — deadlock.

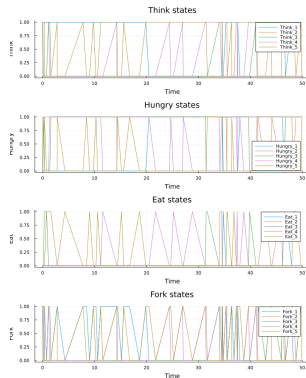


Рисунок 3: Динамика сети с арбитром

Введение позиции Arbiter с N-1 фишками предотвращает блокировку.

Сравнительный анализ состояния «Ест»

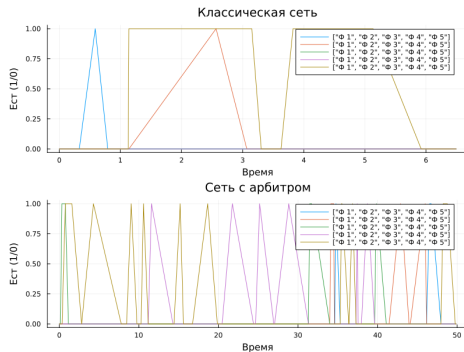


Рисунок 4: Сравнение Eat для обеих моделей

В классической сети все линии падают до нуля; с арбитром — постоянная активность.

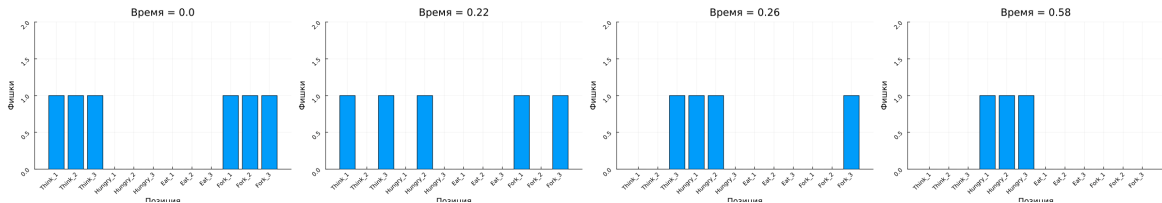
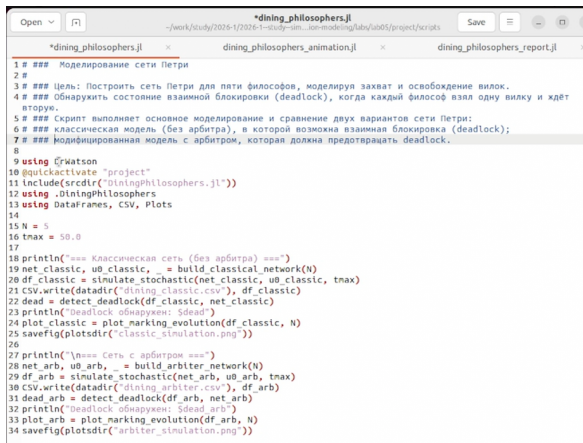


Рисунок 5: Анимация philosophers_simulation.gif

Наглядно показывает перемещение фишек и момент застывания системы.



```
*dining_philosophers.jl
~/work/study/2026-1/2026-1-sim...ion-modeling/lab5/project/scripts Save

*dining_philosophers.jl x dining_philosophers_animation.jl x dining_philosophers_report.jl x

1 # ### Моделирование сети Петри
2 #
3 # ### Цель: Построить сеть Петри для пяти философов, моделируя захват и освобождение вилок.
4 # ### Обнаружить состояние взаимной блокировки (deadlock), когда каждый философ взял одну вилку и ждёт
   вторую.
5 # ### Скрипт выполняет основное моделирование и сравнение двух вариантов сети Петри:
6 # ### классическая модель (без арбитра), в которой возможна взаимная блокировка (deadlock);
7 # ### Модифицированная модель с арбитром, которая должна предотвращать deadlock.
8
9 using Plots, Watson
10 @quickactivate "project"
11 include(scrdir("DiningPhilosophers.jl"))
12 using .DiningPhilosophers
13 using DataFrames, CSV, Plots
14
15 N = 5
16 tmax = 50.0
17
18 println("=== Классическая сеть (без арбитра) ===")
19 net_classic, u0_classic, _ = build_classical_network(N)
20 df_classic = simulate_stochastic(net_classic, u0_classic, tmax)
21 CSV.write(datadir("dining_classic.csv"), df_classic)
22 dead = detect_deadlock(df_classic, net_classic)
23 println("Deadlock обнаружен: $dead")
24 plot_classic = plot_marking_evolution(df_classic, N)
25 savefig(plotsdir("classic_simulation.png"))
26
27 println("\n=== Сеть с арбитром ===")
28 net_arb, u0_arb, _ = build_arbiter_network(N)
29 df_arb = simulate_stochastic(net_arb, u0_arb, tmax)
30 CSV.write(datadir("dining_arbiter.csv"), df_arb)
31 dead_arb = detect_deadlock(df_arb, net_arb)
32 println("Deadlock обнаружен: $dead_arb")
33 plot_arb = plot_marking_evolution(df_arb, N)
34 savefig(plotsdir("arbiter_simulation.png"))
```

Рисунок 6: Пример литературного кода dining_philosophers.jl

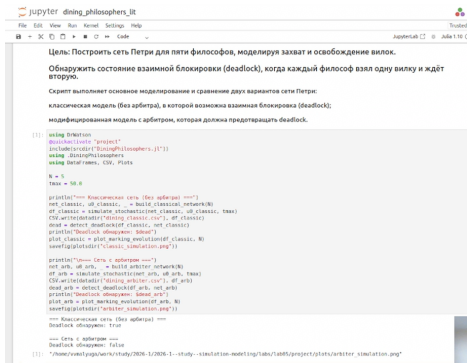

```
Getting /labs/lab05/project/scripts/tangle.jl
vmmalyuga@vmm:~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab05/project$
julia --project=. scripts/tangle.jl scripts/dining_philosophers_lit.jl
Генерация из: scripts/dining_philosophers_lit.jl
[ Info: generating plain script file from `~/work/study/2026-1/2026-1--study--simulation-
modeling/labs/lab05/project/scripts/dining_philosophers_lit.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1--study--simulation-modeling/labs/l
ab05/project/scripts/dining_philosophers_lit/dining_philosophers_lit.jl`
✓ Чистый скрипт: /home/vmmalyuga/work/study/2026-1/2026-1--study--simulation-modeling/l
abs/lab05/project/scripts/../../project/scripts/dining_philosophers_lit/dining_philosoph
ers_lit.jl
[ Info: generating markdown page from `~/work/study/2026-1/2026-1--study--simulation-mod
eling/labs/lab05/project/scripts/dining_philosophers_lit.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1--study--simulation-modeling/labs/l
ab05/project/markdown/dining_philosophers_lit/dining_philosophers_lit.qmd`
✓ Quarto: /home/vmmalyuga/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab
05/project/scripts/../../project/markdown/dining_philosophers_lit/dining_philosophers_li
t.qmd
[ Info: generating notebook from `~/work/study/2026-1/2026-1--study--simulation-modeling
/labs/lab05/project/scripts/dining_philosophers_lit.jl`
[ Info: writing result to `~/work/study/2026-1/2026-1--study--simulation-modeling/labs/l
ab05/project/notebooks/dining_philosophers_lit/dining_philosophers_lit.ipynb`
✓ Notebook: /home/vmmalyuga/work/study/2026-1/2026-1--study--simulation-modeling/labs/l
ab05/project/scripts/../../project/notebooks/dining_philosophers_lit/dining_philosophers
_lit.ipynb

Готово! Все файлы созданы.
vmmalyuga@vmm:~/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab05/project$
```

Рисунок 7: Генерация производных форматов

Из литературных скриптов созданы чистый код, Jupyter Notebook и Quarto-документы.

Проверка в Jupyter Notebook



```
[1]: using OrNSTON
      @using activate "project"
      include("src/dining_philosophers.jl")
      using DiningPhilosophers
      using DataFrames, CSV, Plots

      N = 5
      tmax = 50.0

      println("==== Классическая сеть (без арбитра) ===")
      net_classic, u0_classic, _ = build_classical_network(N)
      et_classic = simulate_stochastic(net_classic, u0_classic, tmax)
      CSV.write{DataFrame}("dining_classic.csv", et_classic)
      dead = detect_deadlock(et_classic, net_classic)
      println("Deadlock observed: $dead")
      plot_classic = plot_marking_evolution(et_classic, N)
      savefig(plotdir("classic_simulation.png"))

      println("==== Сеть с арбитражем ===")
      net_arb, u0_arb, _ = build_arbiter_network(N)
      et_arb = simulate_stochastic(net_arb, u0_arb, tmax)
      CSV.write{DataFrame}("dining_arbiter.csv", et_arb)
      dead_arb = detect_deadlock(et_arb, net_arb)
      println("Deadlock observed: $dead_arb")
      plot_arb = plot_marking_evolution(et_arb, N)
      savefig(plotdir("arbiter_simulation.png"))

      """ Классическая сеть (без арбитра) """
      Deadlock observed: true

      """ Сеть с арбитражем """
      Deadlock observed: false

      [1]: ~/home/vant/papa/work/study/2026-1/2026-1--study--simulation-modeling/labs/lab5/project/plots/arbiter_simulation.png
```

Рисунок 8: Выполнение ноутбука dining_philosophers_lit

Раздел 4

Анализ результатов

- **Классическая сеть:** deadlock неизбежен, через ~10–20 шагов все философы голодны.
- **Сеть с арбитром:** блокировка отсутствует, философы поочерёдно едят.
- **Анимация** визуализирует процесс «застывания» в классической модели.
- **Сравнительный график** количественно подтверждает эффективность арбитра.

Раздел 5

Выводы

- 1 Построены и исследованы две сети Петри для задачи «Обедающие философы».
- 2 Реализовано стохастическое моделирование (алгоритм Гиллеспи).
- 3 Подтверждено возникновение deadlock в классической модели и его предотвращение с помощью арбитра.
- 4 Код оформлен в стиле литературного программирования, сгенерированы все производные форматы.
- 5 Отчёт и презентация подготовлены в Quarto, все результаты воспроизводимы.

Раздел 6

Список литературы



Dijkstra E. W. — Hierarchical ordering of sequential processes. — // *Acta Informatica*. — 1971. — Т. 1, № 2. — С. 115—138.



Petri C. A. — Kommunikation mit Automaten. — *Schriften des Rheinisch-Westfälischen Institutes für instrumentelle Mathematik an der Universität Bonn*, 1962.



Dijkstra E. W. — Hierarchical ordering of sequential processes. — // *Acta Informatica*. — 1971. — T. 1, № 2. — С. 115—138.



Petri C. A. — Kommunikation mit Automaten. — *Schriften des Rheinisch-Westfälischen Institutes für instrumentelle Mathematik an der Universität Bonn*, 1962.